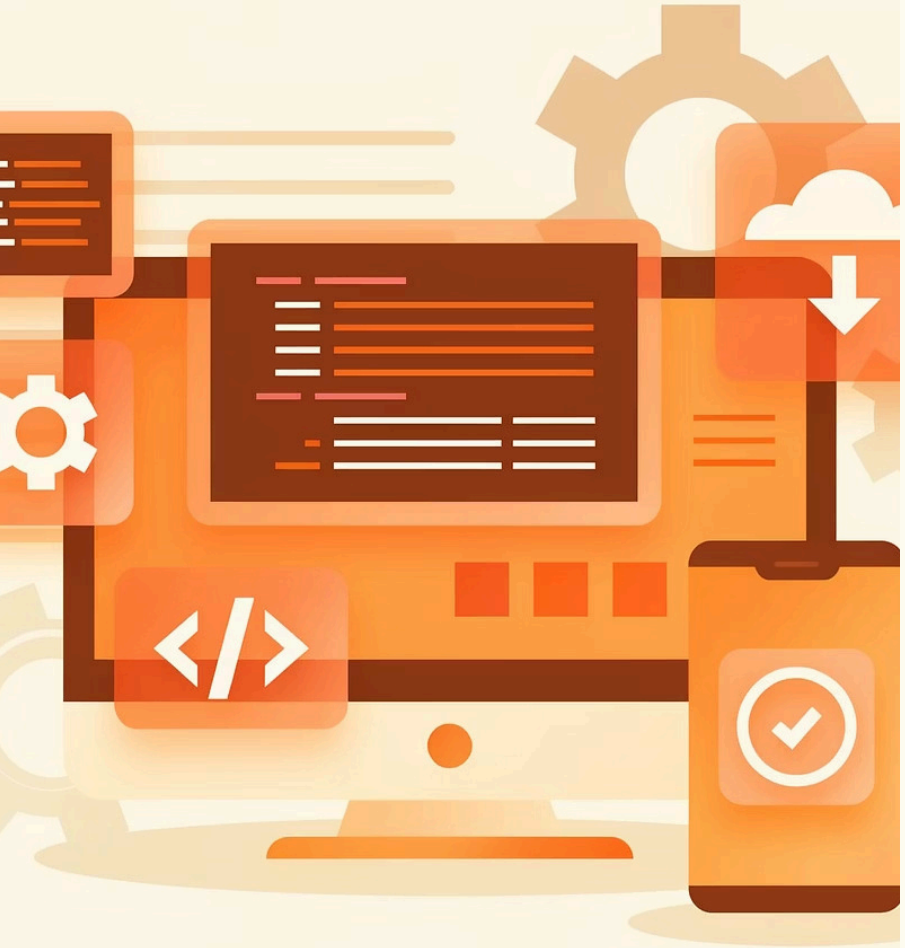


DOM, Navegadores Web y JavaScript

Clase 1 - Introducción

¿Qué es el DOM?

Modelo de árbol

Representación estructurada del documento como un árbol donde cada nodo es una parte del documento.

API para interacción

Interfaz que nos permite cambiar, crear y remover elementos del documento mediante métodos y propiedades.

El navegador lee el HTML y construye el árbol de nodos en memoria, brindando métodos para interactuar con el documento.

Interfaces del DOM

Document Interface

- document.getElementById()
- document.createElement()
- document.querySelector()
- document.querySelectorAll()

Node Interface

- node.parentNode
- node.childNodes
- node.firstChild
- node.nodeType

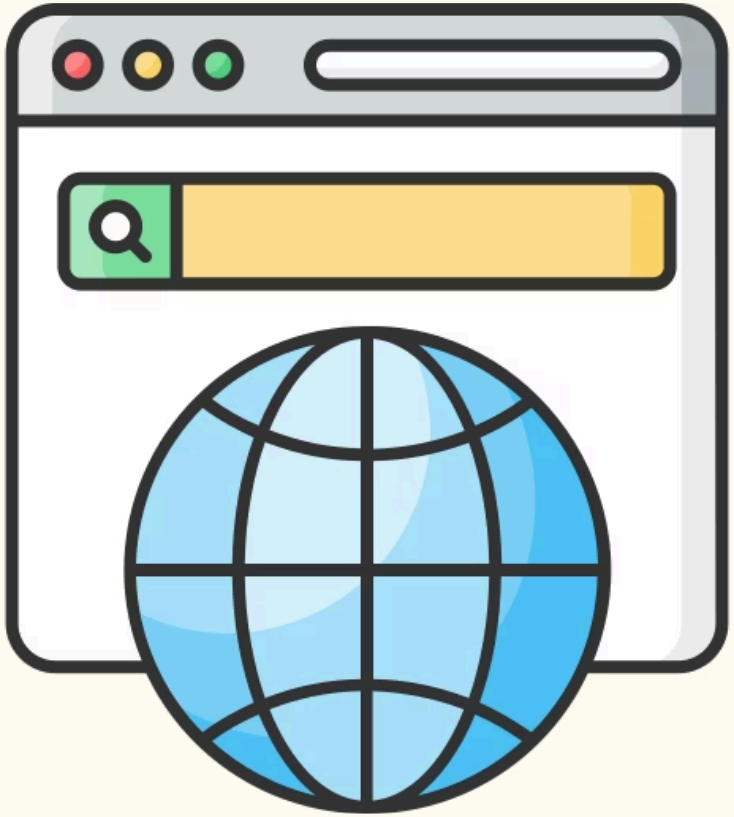
Element Interface

- element.innerHTML
- element.textContent
- element.className
- element.children

Tipos de Nodos

Node: tipo genérico de objeto del DOM, puede ser un elemento, comentario o nodo de texto.

Element: tipo específico de nodo que representa etiquetas HTML. Tienen atributos específicos como id, className y children.



¿Qué es un Navegador Web?

Aplicación de software que permite acceder, visualizar e interactuar con contenido en la World Wide Web. Su función principal es interpretar y renderizar código HTML, CSS y JavaScript.

Proceso de Carga de una Página

01

Resolución DNS

El navegador va al servidor DNS y encuentra la dirección IP real del dominio solicitado.

02

Petición HTTP

El navegador envía una petición HTTP al servidor para obtener la página. Los datos se envían a través de la conexión usando TCP/IP.

03

Respuesta del Servidor

El servidor aprueba la solicitud y envía el mensaje en paquetes que el navegador reúne y procesa.

04

Construcción del DOM

El navegador parsea el HTML y construye el DOM Tree en memoria.

05

Construcción del CSSOM

Se parsea el CSS y construye el CSSOM. El orden de prioridad: navegador, usuario, autor, !important.

06

Render Tree

Se combina el DOM y el CSSOM. Contiene solo elementos visibles con estilos aplicados.

Renderizado: Layout y Paint

Layout (Reflow)

El navegador calcula la posición exacta y el tamaño de cada elemento. Optimizado mediante caché de tamaños y layouts parciales.

Paint

Convierte el render tree en píxeles en la pantalla "pintando" cada elemento según sus propiedades visuales.

Composición

Si hay múltiples capas (animaciones, z-index), el navegador los compone en la imagen final.

Componentes del Navegador

UI (Interfaz de Usuario)

Todo lo que vemos: botones, barras de navegación, pestañas, menús y la ventana principal.

Motor del Navegador

Puente entre UI y motor de renderizado. Parsea HTML y CSS. Chrome/Opera usan Blink, Firefox usa Gecko, Safari usa WebKit.

Motor JavaScript

Interpreta y ejecuta código JS. V8 en Chrome/Opera, SpiderMonkey en Firefox, JavaScriptCore en Safari.

Motor de Red

Maneja solicitudes HTTP/HTTPS y otros protocolos para obtener recursos del servidor.

Almacenamiento de Datos

Gestiona persistencia: cookies, localStorage, sessionStorage, cache y base de datos del navegador.

UI Backend

Se usa para dibujar cajas de selección, ventanas y elementos de interfaz nativas del sistema operativo.

JavaScript

Lenguaje Interpretado y Dinámico

Es un lenguaje interpretado (no compilado), multiparadigma y dinámicamente tipado.

Manipulación del DOM

Es orientado a eventos y permite manipular el DOM, modificando la estructura, contenido y estilos de cualquier documento HTML.

Ejecución en el Navegador

Es ejecutado directamente por los motores JavaScript integrados en los navegadores web. Cuando JS cambia el DOM, el navegador re-renderiza la página.

Conceptos Clave de JavaScript

Variables

`let`, `const` y `var` para declarar variables. `const` permite modificar propiedades de objetos.

Arrow Functions

Sintaxis más corta. Manejan `this` basándose en el bloque donde se definen, no de dónde se llaman.

Desestructuración

Extraer valores de objetos y arrays de manera concisa usando `{}` y `[]`.

Spread Operator

`...` para copiar objetos y arrays sin referencias, o combinar colecciones.

Operador Ternario

Condicionales concisos: `condición ? valor1 : valor2`

Template Literals

Concatenación con backticks: `` `Texto ${variable}` ``

Declaración de Variables

`let`

Se utiliza para declarar variables que pueden ser reasignadas. Tienen alcance de bloque, lo que significa que solo existen dentro del bloque de código donde fueron declaradas.

`const`

Se usa para declarar constantes. Su valor no puede ser reasignado después de la inicialización. Al igual que `let`, tienen alcance de bloque. Para objetos o arrays, las propiedades internas pueden modificarse, pero no la referencia del objeto/array en sí.

`var`

Es la forma tradicional de declarar variables. Tienen alcance de función o global y son susceptibles de "hoisting" (elevación). Su uso se desaconseja en JavaScript moderno debido a comportamientos menos predecibles.

Arrow Functions: Sintaxis y Contexto

Las arrow functions (funciones flecha) ofrecen una sintaxis más concisa para escribir funciones en JavaScript, especialmente útil para funciones anónimas. Una de sus características clave es la forma en que manejan el valor de `this`, que se enlaza al contexto en el que se define la función, no donde se llama.

```
function sumar(a, b) {
  return a + b;
}

const sumar = (a, b) => {
  return a + b;
};

const sumar = (a, b) => a + b;

const duplicar = numero => numero * 2;
```

Desestructuración

Permite extraer valores de arrays u objetos en variables individuales de una manera más concisa y legible. Simplifica la asignación de propiedades de objetos o elementos de arrays a nuevas variables, reduciendo la cantidad de código repetitivo.

Desestructuración de Objetos

```
const usuario = {
  nombre: 'Juan',
  edad: 25,
  ciudad: 'Buenos Aires'
};

// forma tradicional de acceder a propiedades
const nombreTradicional = usuario.nombre;
const edadTradicional = usuario.edad;

// desestructuración básica
const { nombre, edad } = usuario;

// desestructuración con renombrado de variable
const { nombre: nombreUsuario } = usuario;
```

Desestructuración de Arrays

Permite extraer elementos de un array y asignarlos a variables según su posición.

```
const colores = ['rojo', 'verde', 'azul'];

// forma tradicional de acceder a elementos
const primerColorTradicional = colores[0];
const segundoColorTradicional = colores[1];

// desestructuración básica
const [primerColor, segundoColor] = colores;

// saltarse elementos y usar el operador "rest"
const [,, tercerColor, ...restoColores] = colores;
```

Operador Ternario y Spread Operator

Operador Ternario

El operador ternario es una forma concisa de escribir sentencias condicionales `if-else` en una sola línea. Es muy útil para asignaciones de valores o renderizado condicional simple.

```
const edad = 18;
const mensaje = edad >= 18 ? 'mayor de edad' : 'menor de edad';

const nota = 85;
const resultado = nota >= 90 ? 'Excelente' : nota >= 70 ?
'Bueno' : nota >= 50 ? 'Regular' : 'Insuficiente';
```

Spread Operator (...)

Permite expandir elementos de arrays o propiedades de objetos en lugares donde se esperan múltiples elementos o pares clave-valor. Es fundamental para trabajar con inmutabilidad, ya que permite crear copias de arrays y objetos sin mutar los originales.

Ejemplo con Objetos:

```
const book = {
  name: "Just React",
  year: 2022
};

// copia superficial del objeto
const nextBook = { ...book };
nextBook.year = 2023;
console.log(book.year);     // 2022
console.log(nextBook.year); // 2023

// combinar objetos
const authorInfo = { author: "Jane Doe", genre: "Tech" };
const fullBook = { ...book, ...authorInfo };
console.log(fullBook); // { name: "Just React", year: 2022,
author: "Jane Doe", genre: "Tech" }
```

Ejemplo con Arrays:

```
const yearArray = [2019, 2020, 2021];

const newYearsArray = [...yearArray, 2022, 2023];
console.log(newYearsArray); // [2019, 2020, 2021, 2022, 2023]

// concatenar arrays
const moreYears = [2024, 2025];
const allYears = [...newYearsArray, ...moreYears];
console.log(allYears); // [2019, 2020, 2021, 2022, 2023, 2024,
2025]
```

Métodos de Arrays y Eventos

Métodos Inmutables

Estos métodos no modifican el arreglo original, retornan un nuevo array:

- ### map()

  Transforma cada elemento: `numeros.map(num => num * 2)`

- ### filter()

  Filtra elementos: `numeros.filter(num => num % 2 === 0)`

- ### find()

  Encuentra primer elemento que cumple: `usuarios.find(user => user.id === 2)`

- ### reduce()

  Reduce a un valor: `numeros.reduce((acc, num) => acc + num, 0)`

Eventos

Acciones disparadas por interacciones del usuario. Se asocian a elementos específicos o a la ventana.

```
element.addEventListener('click', function() {
 console.log('Click');
});
```